CPE 314 — Software Design

University of Cebu

Computer Engineering Department

Plate N Platter

Plan smarter. Eat better. Cook easier

Project Design Document

Group Name / No.:	PNP/1
Members:	Luistro, Kimberly - Team Leader
	Librea, Nesley Cate - UI
	Nadela, Venice - QA
	Tapan, Yvonne Mae - UI
	Quisagan, Kirk Ryan - Developer
	Tamidles, Ken - DB Admin
Instructor:	Richyll Son
Term:	First Semester, S.Y. 2026-2027
Document Version / Date:	V 0.3 / November 06, 2026

Table of Contents

1. Introduction & Project Overview 5

1. Preliminary ER Sketch & Team Roles 6

1.1 Key attributes and relationships 7

2.2 Team Roles 8

3. System Architecture & Object-Oriented Design 9

3.1 Class Hierarchy 9

3.2 Abstract Class Recipe and the PlanStore Interface 10

3.3 Single Responsibility of Each Class 11

3.4 Design Rationale 13

4. Data Model & Database Design 13

4.1 Database Overview 13

4.2 Tables and Relationship 14

4.3 Core Table Schema 16

4.4 Query Design 17

4.5 Agreement with the ER Sketch 17

5. API Integration Design 18

5.1 The MealDB Call 18

5.2 JSON Response 18

5.3 Parsing 19

5.4 Use in the App 19

6. Concurrency Design 20

6.1 Long Running Operation 20

6.2 Threading Approach 21

6.3 UI Responsiveness 22

6.4 Thread Safety 22

7. Data Mining & Visualization Design 23

7.1 Aggregation Step 23

7.2 Worked Example 24

7.3 Chart and Why It Was Chosen 25

8. Testing Summary..... 26
9. Individual Contributions..... 26
Appendix..... 27
A. References and API Documentation..... 27
B. Licensing Notes..... 27

Revision History

Version	Date	Author	Changes
0.1	October 22, 2026	GROUP 1	Initial draft — Week 8 sections
0.2	October 30, 2026	GROUP 1	Week 9 sections added: Section 3 (System Architecture & OOD) and Section 4 (Data Model & Database Design)
0.3	November 6, 2026	GROUP 1	Week 10 sections added: Section 5 (API Integration), Section 6 (Concurrency) and Section 7 (Data Mining & Visualization). Sections 1, 3, 4, 8 and 9 updated for the Week 10 features: accounts, meal slots, Healthy Levels and prep tiers
1.0			Final compilation and polish — Week 11

1. Introduction & Project Overview

Plate N Platter is a personal meal-planning desktop application for students and busy households, built for Option H (Recipe & Meal Planner) of CPE 314. Each user signs up, logs in, and can change the display name the app greets them with. Users search real recipes from TheMealDB and plan them on a calendar as Breakfast, Lunch, Dinner or Snack. Every planned meal carries its calories and a Healthy Level (Healthy, Moderate or Indulgent), which the app works out from the calories and that meal's budget. Each meal is also tagged Quick, Average or Slow by its prep time. The app builds a shopping list grouped by store aisle, and keeps a planning streak whose badge (Quick Cook, Home Cook or Slow-Food Chef) follows the prep times the user cooks most. Past meals can be imported from a CSV file in the background, and the Analytics tab mines everything into a grouped bar chart. The app solves the everyday problem of deciding what to cook, knowing what to buy, and seeing how healthy one's eating really is, in one simple tool.

Table 2 : Project Summary

Item	Details
Theme	Option H — Recipe & Meal Planner
Target users	Individual users (students, busy households); each user has a private account and sees only their own meals
Platform	Java 17+ desktop application with a JavaFX 17 graphical interface, built with Maven
Data storage	One SQLite file (platenplatter.db) accessed through JDBC, every query with PreparedStatement
Public API	TheMealDB (https://www.themealdb.com/api.php), search by name, shared test key “1”
Key features	Log in, sign up and Account tab; Today by meal slot; Meal Plan calendar colored by Healthy Level; recipe search and planning with calories; nested recipes (recursion); shopping list by aisle; planning streak and prep-tier badge; background CSV import; Analytics grouped bar chart
Repository	https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1 (Week 10 work on project/milestone-2)

1. Preliminary ER Sketch & Team Roles

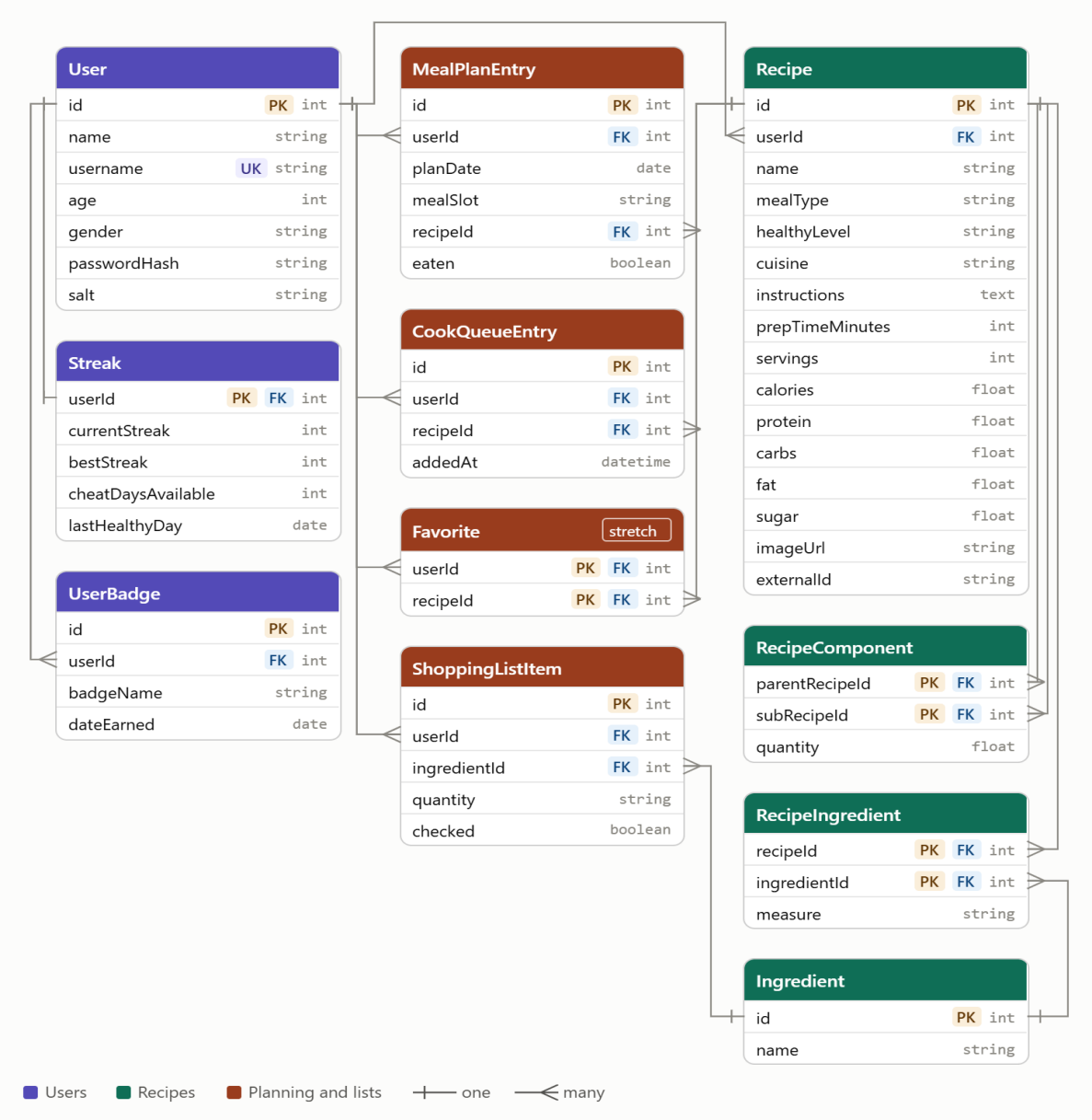


Figure 1: Preliminary ER sketch (entities and key attributes)

1.1 Key attributes and relationships

Table 3: Entities, key attributes, keys and cardinality

Entity	Key attributes	PK / FK / UK	Cardinality
users	name, username, age, gender, password_hash, salt	PK: id UK: username	1 user : M recipes 1 user : M meal_plan_entries 1 user : M meal_history
recipes	name, meal_type, cuisine, healthy_level, prep_minutes, calories, external_id	PK: id FK: user_id → users.id	M recipes : 1 user 1 recipe : M meal_plan_entries 1 recipe : M recipe_components 1 recipe : M recipe_ingredients
recipe_components	quantity	PK: parent_recipe_id + sub_recipe_id FK: parent_recipe_id → recipes.id FK: sub_recipe_id → recipes.id	Join table for recipes M : N recipes (a recipe inside a recipe)
ingredients	name	PK: id UK: name	1 ingredient : M recipe_ingredients
recipe_ingredients	measure	PK: recipe_id + ingredient_id FK: recipe_id → recipes.id FK: ingredient_id → ingredients.id	Join table for recipes M : N ingredients
meal_plan_entries	plan_date, meal_slot, eaten	PK: id FK: user_id → users.id FK: recipe_id → recipes.id	M entries : 1 user M entries : 1 recipe
meal_history	plan_date, meal_slot, recipe_name, category, prep_minutes, calories, healthy_level	PK: id FK: user_id → users.id	M meals : 1 user (the link every tab uses)
meal_ingredient	recipe_name, ingredient	PK: recipe_name + ingredient No FK	M : N meal_history, matched by recipe_name

Table 4: Relationship in the preliminary sketch

From	To	Cardinality	Through and meaning
users	recipes	1 : M	recipes.user_id: a user saves many recipes
users	meal_plan_entries	1 : M	meal_plan_entries.user_id: a user plans many recipe entries
users	meal_history	1 : M	meal_history.user_id: a user owns many planned or imported meals
recipes	meal_plan_entries	1 : M	meal_plan_entries.recipe_id: a recipe can be planned many times
recipes	recipes	M : N	recipe_components: a recipe uses other recipes as sub-recipes
recipes	ingredients	M : N	recipe_ingredients: a recipe has many ingredients and an ingredient is in many recipes
meal_history	meal_ingredient	M : N	recipe_name (no foreign key): a planned dish finds its ingredient list by name

2.2 Team Roles

Table 5:Team members and primary roles

Member	Primary role	Owns in the code
Kimberly Luistro	Team Leader	Recipe hierarchy, UndoStack, CookingQueue, app start-up, AppShell, Analytics tab and chart; reviews and merges pull requests
Nesley Cate Librea	User Interface (UI) Developer	Log in, Sign up, Account, Recipes and Meal Plan screens; stylesheet
Venice Nadela	Quality Assurance (QA) Tester	JUnit 5 test suite, QA test plan, known-issues list
Yvonne Mae Tapan	User Interface (UI) Developer	Header, Today and Import screens; streak and prep-tier badge
Kirk Ryan Quisagan	Software Developer	Recursive RecipeCalculator, healthy-level and prep-tier rules, StatisticsService, shopping list
Kenneth Tamidles	Database Administrator	schema.sql, DatabaseConnector, UserDao, MealPlanHistoryDao, AuthService, PasswordHasher

3. System Architecture & Object-Oriented Design

3.1 Class Hierarchy

Plate N Platter has two class hierarchies (Figure 2): an abstract Recipe with three subclasses, and a PlanStore interface with two implementations. Apart from the start-up class PlateNPlatterApp, every class has one job and sits in one of six packages: model, ui, service, util, db and api.

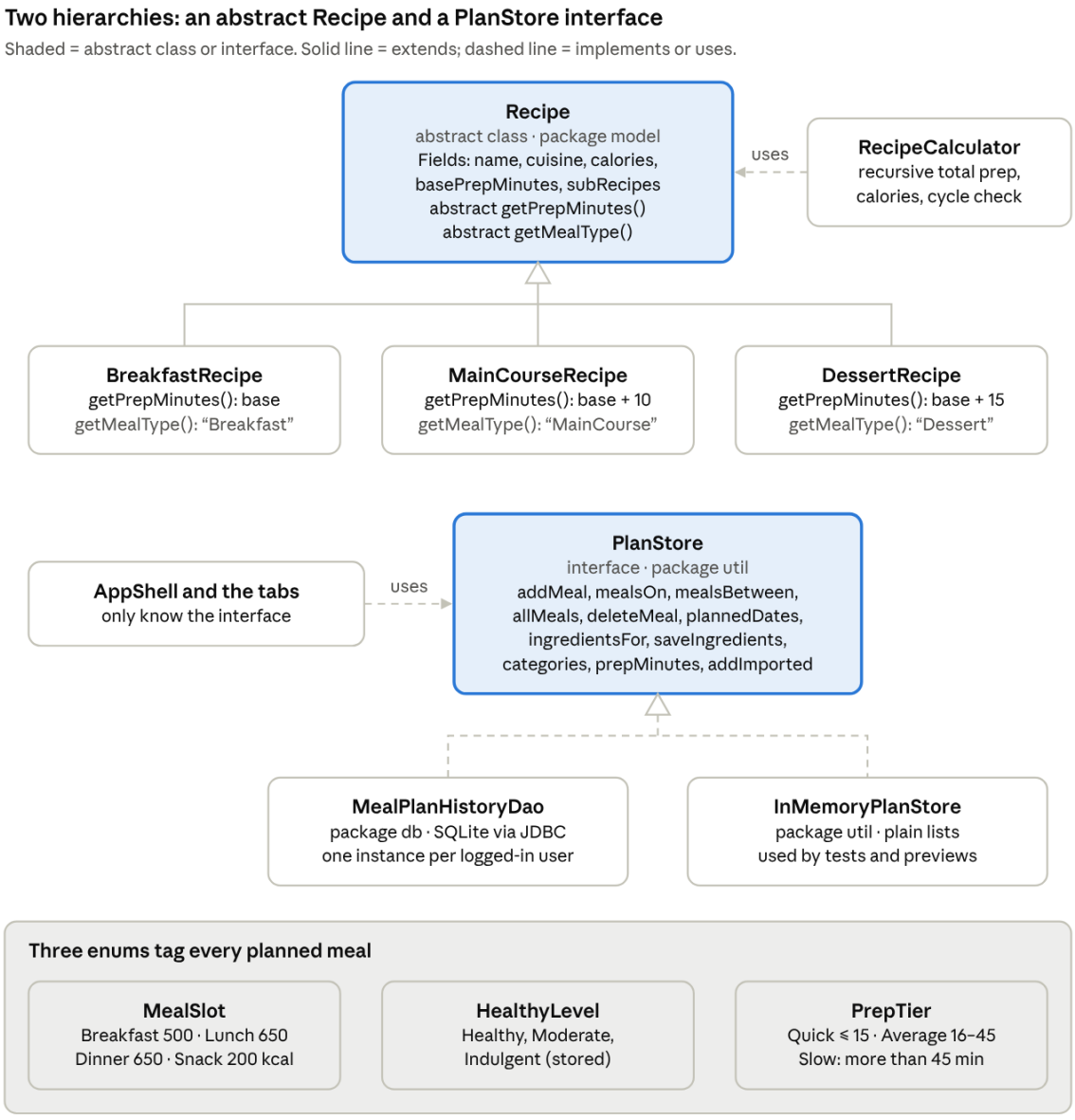


Figure 2: Class hierarchy: abstract Recipe with three subclasses, and the PlanStore interface with two implementations

Every arrow points from a concrete class to the type it extends or implements; the screens and RecipeCalculator depend only on the shaded abstractions. Code that totals prep time calls getPrepMinutes() on a Recipe reference and never checks the subclass, which is polymorphi sm at runtime.

3.2 Abstract Class Recipe and the PlanStore Interface

Abstract class Recipe. Recipe (package model) holds what every recipe has: an id, a name, a cuisine, base prep minutes, calories, and a list of sub-recipes. It declares two abstract methods, getPrepMinutes() and getMealType(). Each subclass adds its own prep-time rule (Table 5), so code that adds up prep time just calls getPrepMinutes() and never checks the type.

Table 6: Recipe subclasses and their prep-time rule

Subclass	getMealType()	getPrepMinutes()	Reason for the extra time
BreakfastRecipe	"Breakfast"	base	none
MainCourseRecipe	"MainCourse"	base + 10	cooking setup
DessertRecipe	"Dessert"	base + 15	chilling or setting

```
public abstract class Recipe {
    protected Recipe(int id, String name, String cuisine, int
basePrepMinutes, int calories) { ... }
    public abstract int getPrepMinutes();   // each subclass adds its own
extra time
    public abstract String getMealType();
    public void addSubRecipe(Recipe sub) { ... }   // a recipe can contain
other recipes
}

public class DessertRecipe extends Recipe {
    @Override
    public int getPrepMinutes() {
        return getBasePrepMinutes() + 15; // dessert: +15 min for
chilling/setting
    }
}
```

Trimmed from model/Recipe.java and model/DessertRecipe.java. Full file: model/Recipe.java

Recursion. RecipeCalculator walks the sub-recipe tree recursively. getTotalPrepMinutes and getTotalCalories add a recipe's own value to the totals of its sub-recipes, to any depth. wouldCreateCycle stops a recipe from being added inside itself, and a loop met while adding up throws CircularRecipeException. In RecipeCalculatorTest, a Cake (Dessert, base 30) holds a Filling (Main course, base 20), which holds a Cream (Breakfast, base 10): total prep = (30 + 15) + (20 + 10) + 10 = 85 minutes.

Interface PlanStore. PlanStore (package util) lists the 11 things a screen can ask of saved meals: add a meal, read the meals of a day, a range or all time, delete a meal, list the planned dates, and save or read a dish's ingredients. MealPlanHistoryDao implements it with SQLite and is created for one logged-in user, so it only sees that user's rows. InMemoryPlanStore implements it with plain lists for tests and previews. AppShell and every screen only know the interface, so they run unchanged on either one.

Enums. MealSlot (Breakfast, Lunch, Dinner, Snack, with budgets of 500, 650, 650 and 200 kcal), HealthyLevel (Healthy, Moderate, Indulgent) and PrepTier (Quick up to 15 min, Average 16 to 45 min, Slow over 45 min) tag every planned meal. The Healthy Level is stored; the prep tier is worked out from the minutes whenever it is needed.

3.3 Single Responsibility of Each Class

Table 7 explains how each class in the two hierarchies follows the Single Responsibility Principle: it has one job and one reason to change. Table 8 does the same for the other main classes.

Table 7: Single Responsibility in the two hierarchies

Class	Kind	Its one responsibility	It changes only when
Recipe	Abstract class (model)	Holds the data every recipe has and declares getPrepMinutes() and getMealType()	the data every recipe carries changes
BreakfastRecipe	Concrete subclass	The breakfast prep-time rule (base minutes)	the breakfast rule changes
MainCourseRecipe	Concrete subclass	The main-course rule (base + 10 minutes)	the main-course rule changes
DessertRecipe	Concrete subclass	The dessert rule (base + 15 minutes)	the dessert rule changes
PlanStore	Interface (util)	Lists what a screen may ask of saved meals; holds no storage code	screens need a new kind of question
MealPlanHistoryDao	Implementation (db)	All SQL for meal_history and meal_ingredient, through PreparedStatement, for one user	the tables or the SQL change
InMemoryPlanStore	Implementation (util)	Keeps meals in plain lists for tests and previews	tests need new fake behavior

Table 8: Main classes and their single responsibility

Package	Class	Its one job
(root)	PlateNPlatterApp	Starts the app, opens the database, shows Log in, then opens the tabs for the logged-in user
ui	AppShell, HeaderBar	Builds the seven tabs; the header shows the greeting, clock, streak, badge and Log out
ui	LoginScreen, RegisterScreen, AccountScreen	Log in, sign up, change the display name
ui	TodayScreen, MealPlanScreen, RecipesScreen, ShoppingListScreen, ImportScreen, AnalyticsScreen	One tab each: show data and pass clicks on to the logic classes
ui	CsvImportTask	Imports a CSV file on a background thread and reports progress
ui	AnalyticsChart	Builds the labeled grouped BarChart
service	AuthService	Sign-up, log-in and rename rules
service	StatisticsService	Counts and averages for the Analytics tab
util	CsvRecipeParser	Turns one CSV line into an ImportRow, or rejects it with a reason
util	HealthClassifier	Works out the Healthy Level from calories, meal slot and category
util	CategoryMapper	Maps a TheMealDB category to Breakfast, Main Course or Dessert
util	StreakCalculator	Current and best streak, and the prep-tier badge
util	ShoppingPlanner, ShoppingListBuilder	Collects the ingredients of a date range and groups them by aisle
util	RecipeCalculator	Recursive totals and the cycle check for nested recipes
util	PasswordHasher	Salted PBKDF2 password hashes
util	DataChangeNotifier	Tells every tab to refresh after a change (Observer pattern)
db	UserDao, MealPlanHistoryDao	All SQL for users and meals, always through PreparedStatement
db	DatabaseConnector	Opens connections with foreign keys on and a 5-second busy timeout

Package	Class	Its one job
api	MealDbClient	Calls TheMealDB and parses the JSON into MealData

3.4 Design Rationale

`Recipe` is abstract because there is no plain recipe: every recipe is one of three types, and the type changes how long it takes. `PlanStore` is an interface so the screens depend on what they need, not on SQLite; the previews and tests swap in `InMemoryPlanStore` with no database file. The screens never write SQL and the DAOs never touch JavaFX, so a change to the database or to the look stays inside one package. If TheMealDB changes its JSON, only `MealDbClient` changes.

4. Data Model & Database Design

4.1 Database Overview

The final database is one SQLite file, `platenplatter.db`, with 8 tables defined in `src/main/resources/schema.sql` (Figure 3). The Milestone 2 app reads and writes three of them: `users`, `meal_history` and `meal_ingredient`. The other five hold the full recipe model from the Week 8 sketch; they are created, but the tabs do not use them yet.

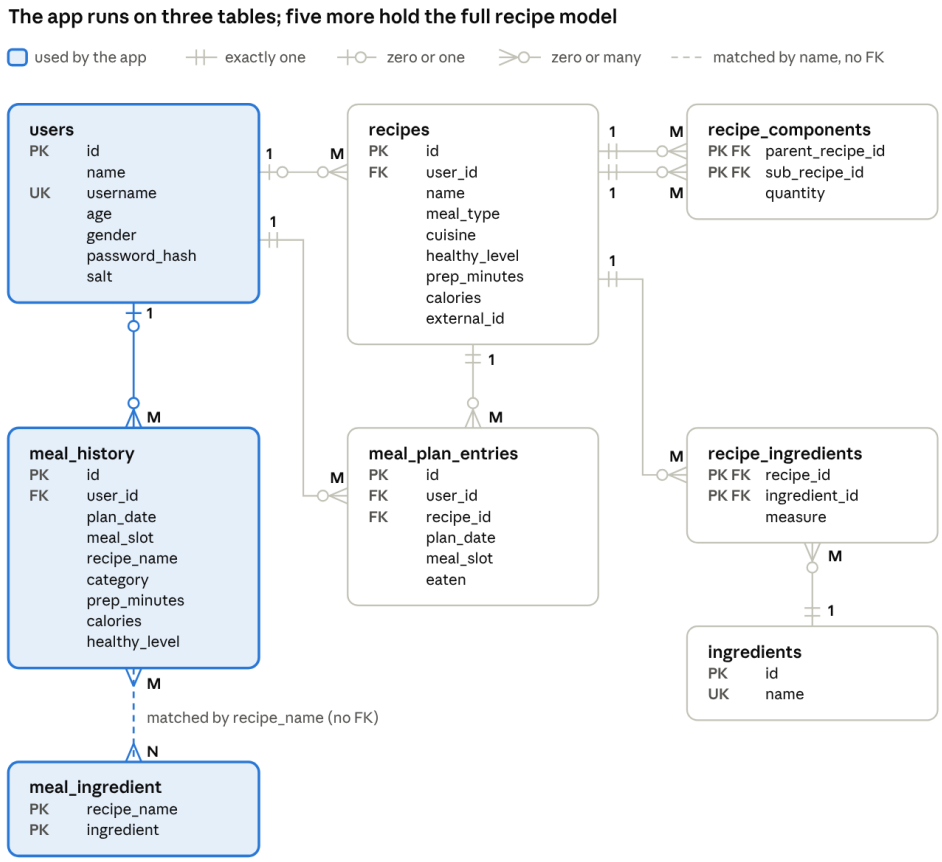


Figure 3: Final ER diagram of platenplatter.db (shaded tables are the ones the app uses)

The shaded tables and lines are what the app uses today: one user owns many rows in `meal_history`, and each dish's ingredients are found in `meal_ingredient` by its name.

4.2 Tables and Relationship

Table 9: The 8 tables in schema.sql

Table	Columns	Keys and rules	Used by the app
users	id, name, username, age, gender, password_hash, salt	PK id; username NOT NULL UNIQUE; name, password_hash, salt NOT NULL	Yes: log in, sign up, Account
recipes	id, user_id, name, meal_type, cuisine, healthy_level, prep_minutes, calories, external_id	PK id; FK user_id → users(id); prep_minutes and calories default 0	Not yet

Table	Columns	Keys and rules	Used by the app
recipe_components	parent_recipe_id, sub_recipe_id, quantity	PK (parent_recipe_id, sub_recipe_id); both FK → recipes(id); quantity default 1	Not yet
ingredients	id, name	PK id; name NOT NULL UNIQUE	Not yet
recipe_ingredients	recipe_id, ingredient_id, measure	PK (recipe_id, ingredient_id); FK → recipes(id) and ingredients(id)	Not yet
meal_plan_entries	id, user_id, recipe_id, plan_date, meal_slot, eaten	PK id; FK user_id → users(id), recipe_id → recipes(id); eaten default 0	Not yet
meal_history	id, user_id, plan_date, meal_slot, recipe_name, category, prep_minutes, calories, healthy_level	PK id; FK user_id → users(id); CHECK rules on meal_slot, healthy_level, prep_minutes, calories	Yes: every tab
meal_ingredient	recipe_name, ingredient	PK (recipe_name, ingredient); recipe_name COLLATE NOCASE	Yes: Recipes saves, Shopping List reads

Table 10: Relationships and cardinality in the final schema

From	To	Cardinality	Through and meaning
users	recipes	1 : M	recipes.user_id: a user saves many recipes
users	meal_plan_entries	1 : M	meal_plan_entries.user_id: a user plans many recipe entries
users	meal_history	1 : M	meal_history.user_id: a user owns many planned or imported meals
recipes	meal_plan_entries	1 : M	meal_plan_entries.recipe_id: a recipe can be planned many times
recipes	recipes	M : N	recipe_components: a recipe uses other recipes as sub-recipes
recipes	ingredients	M : N	recipe_ingredients: a recipe has many ingredients and an ingredient is in many recipes
meal_history	meal_ingredient	M : N	recipe_name (no foreign key): a planned dish finds its ingredient list by name

4.3 Core Table Schema

meal_history is the table every tab reads and writes; the other seven CREATE TABLE statements are in schema.sql.

```
CREATE TABLE IF NOT EXISTS meal_history (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER REFERENCES users(id),
    plan_date TEXT NOT NULL,
    meal_slot TEXT NOT NULL DEFAULT 'DINNER'
        CHECK (meal_slot IN ('BREAKFAST', 'LUNCH', 'DINNER', 'SNACK')),
    recipe_name TEXT NOT NULL,
    category TEXT NOT NULL,
    prep_minutes INTEGER NOT NULL CHECK (prep_minutes >= 0),
    calories INTEGER NOT NULL DEFAULT 0 CHECK (calories >= 0),
    healthy_level TEXT NOT NULL DEFAULT 'MODERATE'
        CHECK (healthy_level IN ('HEALTHY', 'MODERATE', 'INDULGENT'))
);
```

From src/main/resources/schema.sql. Full file: schema.sql

- **Rules live in the database too.** meal_slot and healthy_level only accept their enum names, and prep_minutes and calories cannot be negative, so SQLite rejects a bad row even if a Java check is skipped.
- **Foreign keys are on.** DatabaseConnector runs PRAGMA foreign_keys = ON for every connection, so a meal cannot belong to a user who does not exist.
- **Passwords are never stored.** users keeps a salted PBKDF2-HMAC-SHA256 hash (65,536 rounds) and the salt.
- **Old files are upgraded.** A platenplatter.db made before Week 10 lacks the four new columns. On start-up, MealPlanHistoryDao.initSchema() adds each missing column with ALTER TABLE and keeps the old meals; meals saved before accounts existed go to the first user who logs in.
- **The prep tier is not a column.** Quick, Average or Slow is worked out from prep_minutes each time, so it can never disagree with the minutes.

4.4 Query Design

Every query uses PreparedStatement and is scoped to one user. The DAO made at log-in holds that user's id, and every query adds (? IS NULL OR user_id = ?):

```
private static final String MINE = "(? IS NULL OR user_id = ?)";
static final String SELECT_BETWEEN = "SELECT " + COLUMNS
       + " FROM meal_history WHERE plan_date BETWEEN ? AND ? AND "
        + MINE + " ORDER BY plan_date, " + SLOT_ORDER + ", id";

private List<PlannedMeal> queryMeals(String from, String to)
        throws SQLException {
    try (PreparedStatement ps = conn.prepareStatement(SELECT_BETWEEN)) {
        ps.setString(1, from);
        ps.setString(2, to);
       bindMine(ps, 3);   // fills both ? with this DAO's user id
       return readMeals(ps);
    }
}
```

Condensed from db/MealPlanHistoryDao.java. Full file: db/MealPlanHistoryDao.java

4.5 Agreement with the ER Sketch

Figure 1, Figure 3, Table 3 and Table 9 list exactly the same 8 tables, columns and keys, and Table 4 and Table 10 list the same relationships. Compared with the very first Week 8 sketch, the changes are:

- Streaks and badges are worked out from the meal dates, so there are no Streak or UserBadge tables.
- The shopping list is built for a date range when it is opened, so there is no ShoppingListItem table.
- The cook queue and undo stack live in memory (CookingQueue, UndoStack), so they have no tables. Favorites, a stretch goal, was dropped.
- meal_history was added: one flat row per planned or imported meal, so the calendar, streak and chart count both with one query. meal_ingredient was added to remember each dish's ingredients by name for the shopping list.

5. API Integration Design

5.1 The MealDB Call

Plate N Platter consumes TheMealDB, a free public REST API, through one endpoint: search by meal name. MealDbClient (package api) makes the call and turns the JSON into MealData objects, which the Recipes tab and the Shopping List use (Table 11).

Table 11: The TheMealDB call

Item	Detail
Endpoint	GET https://www.themealdb.com/api/json/v1/1/search.php?s={name} (the name is URL-encoded)
Key	The shared free test key 1, which is part of the path; no sign-up or login
Client	Java HttpClient, with the header User-Agent: PlateNPlatter/1.0
Success	HTTP 200 and a JSON object
Failure	Any other status throws an IOException; so does a missing network
Called from	RecipesScreen (the search box) and IngredientLookup (“Look up online” on the Shopping List)

5.2 JSON Response

The response is one object with a meals array. Each meal is a flat object of strings: idMeal, strMeal, strCategory, strArea, strInstructions, and twenty numbered slots strIngredient1 to strIngredient20, plus fields we do not read, such as strMealThumb and strMeasure1 to strMeasure20. Unused ingredient slots are empty strings or null. When nothing matches, meals is null, not an empty array.

```
{
"meals": [
  {
        "idMeal": "52771",
        "strMeal": "Spicy Arrabiata Penne",
        "strCategory": "Vegetarian",
        "strArea": "Italian",
        "strInstructions": "Bring a large pot of water to a boil...",
        "strIngredient1": "penne rigate",
        "strIngredient2": "olive oil",
        "strIngredient9": "",
        "strIngredient16": null
    }
  ]
}
```

Trimmed: a real meal has all twenty ingredient slots.

5.3 Parsing

MealDbClient.parseMeals reads the JSON with Gson. It is public and static, so the tests run it without internet. It returns an empty list for "meals": null, reads five text fields, and keeps only the ingredient slots that are not blank.

```
public static List<MealData> parseMeals(String json) {
    List<MealData> result = new ArrayList<>();
    JsonObject root = JsonParser.parseString(json).getAsJsonObject();
    JsonElement mealsElement = root.get("meals");
    if (mealsElement == null || mealsElement.isJsonNull()) {
        return result;  // {"meals":null} when nothing matches
    }
    for (JsonElement e : mealsElement.getAsJsonArray()) {
        JsonObject m = e.getAsJsonObject();
        List<String> ingredients = new ArrayList<>();
        for (int i = 1; i <= 20; i++) {
            String ing = text(m, "strIngredient" + i); // "" if null
            if (!ing.isBlank()) ingredients.add(ing.trim());
        }
        result.add(new MealData(text(m, "idMeal"), text(m, "strMeal"),
                text(m, "strCategory"), text(m, "strArea"),
                text(m, "strInstructions"), ingredients));
    }
    return result;
}
```

Condensed from api/MealDbClient.java. Full file: [api/MealDbClient.java]

5.4 Use in the App

Table 12: How the parsed data is used in the app

Field	How the app uses it
strMeal	The results list on Recipes; saved as meal_history.recipe_name when the meal is planned
strCategory	CategoryMapper turns it into our category (Breakfast, Dessert or Main Course), which picks the default meal slot. With no calories typed, HealthClassifier uses it for the Healthy Level: Vegetarian, Vegan and Seafood are Healthy, Dessert is Indulgent, the rest Moderate
strArea	The cuisine shown under each result and as a chip on the recipe
strInstructions	The recipe text on Recipes
strIngredient1 to 20	Saved to meal_ingredient with the meal; the Shopping List groups them by aisle

Field	How the app uses it
(not in the API)	TheMealDB gives no calories or prep time, so the user enters them; the Auto Healthy Level is then worked out from the calories and the meal slot's budget

Error handling.

- No internet, or a status other than 200: the search Task fails and `setOnFailed` shows "Could not reach TheMealDB. Check your internet connection." Everything already saved keeps working.
- No match: `"meals": null` becomes an empty list, and the status line says "No recipes found for "…". Try another word."
- Missing fields: a missing or `null` field becomes empty text, blank ingredient slots are skipped, and a blank category falls back to Main Course.
- Wrong dish: "Look up online" saves ingredients only for an exact name match, so a different dish's ingredients are never attached.
- Double clicks: the Search button is disabled while a search runs.

To prove the call is live during the demo, turn Wi-Fi off and search (the error message appears), then turn it back on and search again (real results).

6. Concurrency Design

6.1 Long Running Operation

Anything that can take more than a moment runs off the JavaFX Application Thread: reading a big file, calling the internet, querying SQLite, and hashing a password. Table 13 lists every such operation and what the user sees while it runs. The largest one is the CSV import: the sample files sample_data/meal_history_3000.csv and meal_history_30000.csv hold 3,000 and 30,000 rows.

Table 13: Operations that run on a background thread

Operation	Where	Work done in call()	What the user sees
Import a meal-history CSV	CsvImportTask, started by ImportScreen	Read the lines, parse each one, save it through PlanStore.addImported	Progress bar and "Imported n rows, skipped k" every 50 rows; Cancel button
Search TheMealDB	RecipesScreen	MealDbClient.searchByName: HTTP call and JSON parsing	"Searching…", Search disabled, then results or the error message

Operation	Where	Work done in call()	What the user sees
Save a planned meal	RecipesScreen	PlanStore.addMeal with its ingredient list	Add button disabled until saved; other tabs refresh
Log in, sign up, rename	LoginScreen, RegisterScreen, AccountScreen	AuthService: PBKDF2 hashing (65,536 rounds) and SQL	Button disabled so it cannot run twice; status message
Claim meals saved before accounts	PlateNPlatterApp.openAppFor	MealPlanHistoryDao.claimUnownedMeals	The tabs open when it finishes
Load Today, the calendar and the streak	TodayScreen, MealPlanScreen, HeaderBar	mealsOn, mealsBetween, plannedDates, StreakCalculator	The data appears when ready
Build the shopping list	ShoppingListScreen	ShoppingPlanner.plan over a date range	The list appears when ready
Mine the data for the chart	AnalyticsScreen	mealsBetween or allMeals, then StatisticsService	“Loading…”, Refresh disabled, then the chart

6.2 Threading Approach

Each operation is a javafx.concurrent.Task started on its own daemon Thread (new Thread(task), setDaemon(true), start()). A daemon thread never keeps the app alive after the window closes. The rule the code follows is simple: call() does the slow work and never touches the screen, while setOnSucceeded, setOnFailed and setOnCancelled run back on the JavaFX Application Thread and update the screen. Figure 4 shows the CSV import end to end.

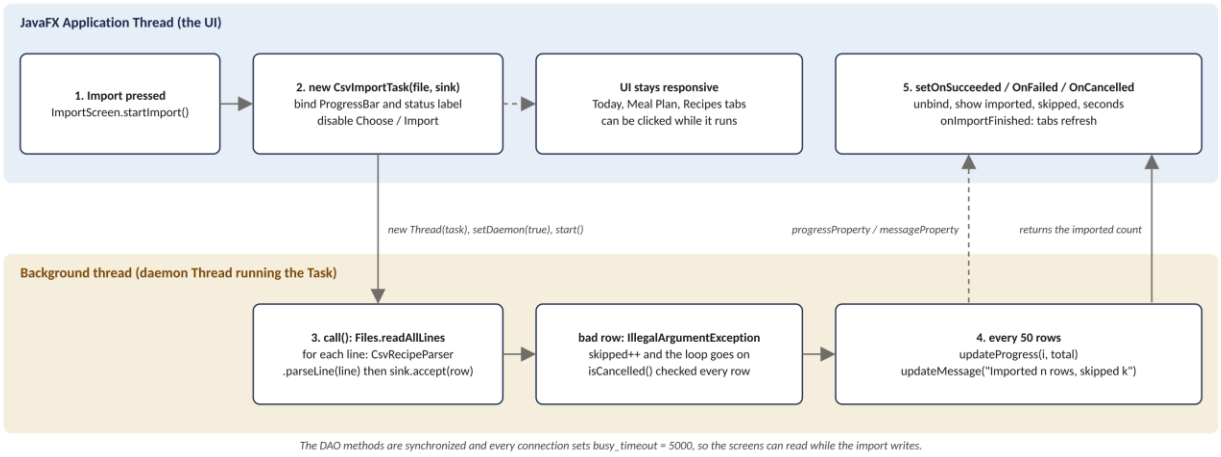


Figure 4: The CSV import: the UI thread starts the Task and stays free while the background thread reads the file

```
CsvImportTask task = new CsvImportTask(chosenFile, sink);
bar.progressProperty().bind(task.progressProperty());  // bar follows Task
statusLabel.textProperty().bind(task.messageProperty());
task.setOnSucceeded(e -> finish(task, "Done! The import finished.",
        1.0, Theme.Kind.OK, true));
task.setOnFailed(e -> finish(task, "Import failed: "
        + task.getException().getMessage(), 0, Theme.Kind.ERROR, false));
task.setOnCancelled(e -> finish(task, "Import cancelled.",
        0, Theme.Kind.INFO, true));
Thread thread = new Thread(task);  // BACKGROUND THREAD: UI stays
responsive
thread.setDaemon(true);
thread.start();
```

Condensed from ui/ImportScreen.java (startImport). Full file: ui/ImportScreen.java

6.3 UI Responsiveness

How the UI stays responsive:

- The window never waits: while a 30,000-row file is imported, the user can click Today, Meal Plan or Recipes, and the clock in the header keeps ticking.
- Progress is reported, not pushed row by row: CsvImportTask calls updateProgress and updateMessage only every 50 rows, so the UI thread is not flooded.
- Bad rows do not stop the import: CsvRecipeParser throws IllegalArgumentException for a bad line, the Task counts it as skipped and carries on. Cancel works because call() checks isCancelled() before each row.
- Buttons that start work are disabled while it runs, so a double click cannot start it twice.
- When an import or a save finishes, DataChangeNotifier.fire() is called on the UI thread and every tab refreshes (Observer pattern).

6.4 Thread Safety

- The MealPlanHistoryDao methods are synchronized, because the import writes while the screens read through the same connection.
- Every connection sets PRAGMA busy_timeout = 5000, so a second connection waits up to 5 seconds instead of failing with “database is locked”.
- Screen objects are only changed in setOnSucceeded and setOnFailed, never inside call().

7. Data Mining & Visualization Design

7.1 Aggregation Step

The Analytics tab mines the user's own meals from SQLite. A background Task reads the meals of the chosen week (mealsBetween, Monday to Sunday) or of all time (allMeals). StatisticsService then aggregates them (Table 14). countByDay first builds a table of 7 days × every group, all starting at 0, so an empty day still shows; then it adds 1 per meal with Map.merge. The user picks what the bars are split by: Healthy Level, prep tier or meal slot.

Table 14: Data-mining methods in StatisticsService

Method	What it computes	Shown as
countByDay(rows, monday, grouping)	For one week: day → (group → number of meals); meals outside the week are ignored	The grouped bar chart (Week view)
countByWeekday(rows, grouping)	For all time: Mon…Sun → (group → number of meals)	The grouped bar chart (All time view)
totals(rows, grouping)	Number of meals per Healthy Level, prep tier and meal slot	The breakdown panel with percentages
healthyShare(rows)	Healthy meals ÷ all meals (NaN when there are none)	The "Healthy share" card
averageCalories(rows), averagePrepMinutes(rows)	Averages; calories only over meals whose calories are known	The "Average calories" and "Average prep time" cards
levelByDay(rows)	The level most of a day's meals have; a tie goes to the less healthy level	The color of each day on the Meal Plan calendar

```
public static Map<String, Map<String, Integer>> countByDay(
        List<PlannedMeal> rows, LocalDate monday, Grouping grouping) {
    Map<String, Map<String, Integer>> table = new LinkedHashMap<>();
    for (int d = 0; d < 7; d++) {   // 7 days x every group, all at 0
        table.put(dayLabel(monday.plusDays(d)), zeros(grouping));
    }
    LocalDate sunday = monday.plusDays(6);
    for (PlannedMeal meal : safe(rows)) {
        LocalDate date = dateOf(meal);
        if (date != null && !date.isBefore(monday)
                && !date.isAfter(sunday)) {
            table.get(dayLabel(date))
                 .merge(grouping.seriesOf(meal), 1, Integer::sum);
        }
    }
    return table;
}
```

From service/StatisticsService.java. Full file: service/StatisticsService.java

7.2 Worked Example

Importing sample_data/meal_history_week10.csv (48 meals over 2 to 15 November 2026) and opening the week of 2 November gives the counts in Table 15: 24 meals, of which 14 are Healthy (a healthy share of 58%), an average of 532.5 kcal (shown on the card as 533 kcal) and 37.5 minutes of prep per meal.

Table 15: Mined counts for the week of 2 November 2026 (Healthy Level grouping)

Group	Mon 2	Tue 3	Wed 4	Thu 5	Fri 6	Sat 7	Sun 8	Total
Healthy	2	3	2	1	2	2	2	14
Moderate	1	1	0	2	1	0	1	6
Indulgent	0	0	1	0	1	1	1	4
All meals	3	4	3	3	4	3	4	24

7.3 Chart and Why It Was Chosen

AnalyticsChart.buildGrouped draws one series per group, with one bar per day: the title “Meals Planned per Day by …” with the week or “All Time”, the x-axis labeled “Day”, the y-axis labeled “Number of meals planned”, and a legend. Healthy is always green, Moderate yellow and Indulgent red, the same colors as the calendar and the meal chips. Figure 5 shows the chart for the counts in Table 15.

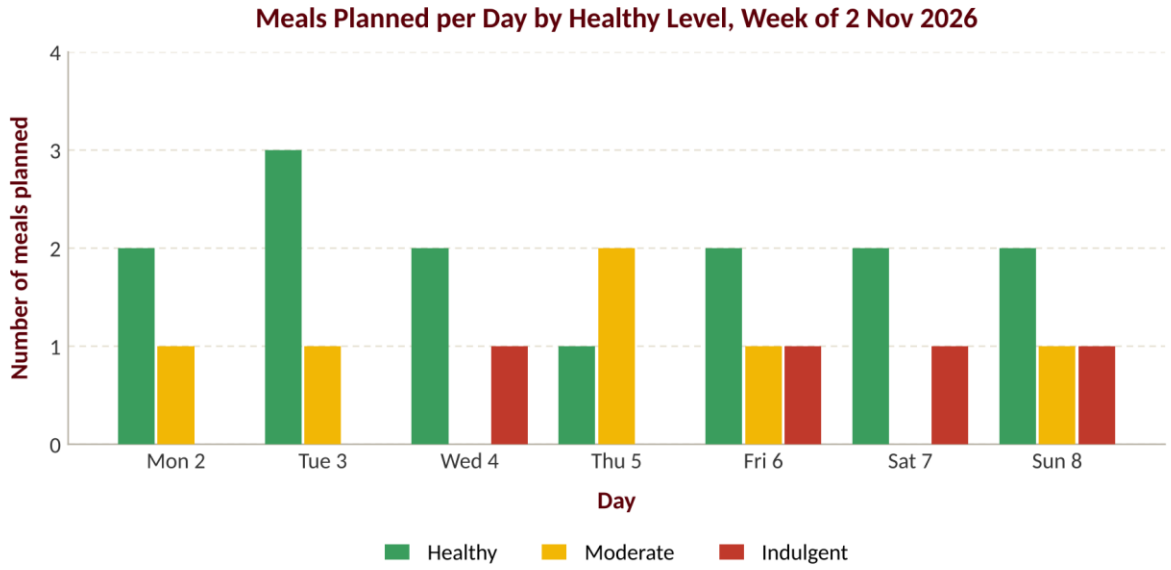


Figure 5: Grouped bar chart of the mined counts for the week of 2 November 2026

Why a grouped bar chart? Days and groups are categories and the counts are small whole numbers, so bars compare them best, and placing the groups side by side shows each day’s mix at a glance. A line chart would suggest smooth change between days, and a pie chart shows only one total, so a week would need seven pies.

The result is not hard-coded. Removing or adding a meal and pressing Refresh changes the bars. StatisticsServiceTest.changingAMealsLevelChangesTheMinedResult and MealPlanHistoryDaoTest.changingTheDataChangesTheMinedResult prove it automatically.

8. Testing Summary

Summarize your JUnit test suite — how many tests exist, what they cover, and at least one meaningful edge case per tested class. A short table or list of key test cases is helpful here.

9. Individual Contributions

List what each team member specifically built or owned, referencing their git commit history, so the instructor can cross-check contribution during the Individual Q&A.

Appendix

A. References and API Documentation

- TheMealDB API documentation: https://www.themealdb.com/api.php
- Project repository: https://github.com/Kimonooo/cpe314-69167-capstoneteam-Meal_plannerGroup1
- OpenJFX (JavaFX) documentation: https://openjfx.io/
- SQLite documentation (foreign keys, CHECK, PRAGMA): https://www.sqlite.org/docs.html
- Xerial SQLite JDBC driver: https://github.com/xerial/sqlite-jdbc
- Gson JSON library: https://github.com/google/gson
- JUnit 5 User Guide: https://junit.org/junit5/docs/current/user-guide/

B. Licensing Notes

- TheMealDB: the test key 1 may be used during development or for educational use; a supporter key is required to release an app publicly on an app store. Recipe text and data belong to TheMealDB.
- Libraries: OpenJFX 17 (GPL v2 with the Classpath Exception), sqlite-jdbc 3.45 (Apache License 2.0), Gson 2.10 (Apache License 2.0), JUnit 5.10 (Eclipse Public License 2.0).
- The sample CSV files in sample_data/ were made by the team for testing and the demo.